

12

PROCESS INJECTION, MANIPULATION, AND HOOKING



To blend in with their target environment, modern evasive threats must remain hidden on the infected host. Two methods they use to do so are process injection and process image manipulation. *Process injection* involves injecting and executing nefarious code inside another process rather than executing it directly, while *process image manipulation* involves tampering with process images and abusing the way Windows handles processes. Malware can also use process injection techniques to inject hooks into a target process. Hooking allows the malware to intercept API function calls and monitor or manipulate them, helping it remain undetected.

We'll begin this chapter by looking at different forms of process injection. Then, we'll discuss two close cousins of the technique (process image manipulation and DLL and shim hijacking) along with various hooking methods. At the end of the chapter, we'll briefly explore how to mitigate these types of attacks.

Process Injection

There are several reasons why malware might want to implement process injection techniques:

Hiding from defenses and investigators

Injecting code into another process, particularly a well-known one like *notepad.exe* or *explorer.exe*, may help the malware persist on the infected host, hidden from endpoint defenses and investigators.

Mimicking normal behavior

Malware might inject code into certain processes to disguise its behavior. For example, injecting code into a web browser and communicating with a C2 server from this process can help hide suspicious web traffic, as it's normal and expected for web browsers to communicate with the internet.

Thwarting debugging efforts

Injecting code into a remote process can help circumvent and evade tools like debuggers, causing the analyst to lose control of the malware's code execution flow.

Elevating privileges

Specific types of process injection techniques can help malware elevate its privileges on the infected host, giving it a higher level of access within the system.

Intercepting data via hooking

Injecting hooking code into a process can allow the malware to intercept and modify Windows API calls or intercept sensitive data.

Let's examine various process injection techniques in more detail, starting with how malware identifies a target process for injection.

Random vs. Specific Target Processes

Malware can inject code into either a random process or a specifically chosen target, depending on what it's trying to achieve. For example, some malware injects malicious code into multiple arbitrary processes on the host to ensure its own survival. The malware could simply enumerate all processes on the host using `Process32First` and `Process32Next` and then attempt to open and get a handle to the target process using `OpenProcess`. If it successfully opens the target process with its current privilege levels, the malware injects its code there. This approach isn't very stealthy, however. A more covert approach is to inject code into specific target processes, such as well-known and common Windows processes, or into processes that allow the malware to achieve a certain objective. Certain variants of the malware family Formbook, for example, inject code into browser-related processes, attempting to sniff out sensitive data such as web logins.

Some malware might even inject code into its own process (a technique known as *self-injection*) or into a child process that it creates. These types of injection techniques often take place during the *unpacking* process, in which malware decrypts, or unpacks, its payload in memory and then in-

jects it into a child process. [Chapter 17](#) will discuss unpacking in more detail. For now, let's look at one of the most basic and prevalent forms of process injection: shellcode injection.

Shellcode Injection

Shellcode injection, also known as *direct injection*, is one of the oldest injection techniques; as its name suggests, it involves injecting *shellcode*, a type of position-independent machine code. Once the shellcode has been injected, the malware can remain hidden while executing its malicious code directly from within the victim process's memory. [Figure 12-1](#) illustrates how it works.

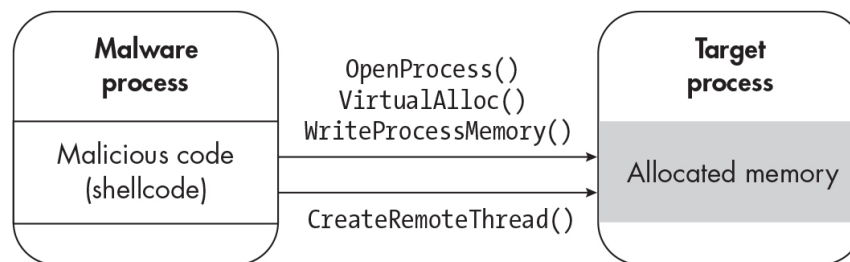


Figure 12-1: The shellcode injection technique

To inject the shellcode, first the malware must open an object handle to the target process by calling `OpenProcess` (or by directly calling its native API equivalent, `NtOpenProcess`). The `OpenProcess` function has a couple of important parameters: `dwDesiredAccess` and `dwProcessId`. The `dwDesiredAccess` parameter represents the access rights requested by the calling process, and `dwProcessId` is the ID of the target process. You can reference the function prototype information for `OpenProcess` like so:

```

OpenProcess(
    DWORD dwDesiredAccess, // Access rights requested.
    BOOL bInheritHandle,   // If true, processes created by this process inherit this process
    DWORD dwProcessId      // Process ID of the target process
);
  
```

Once the malware has obtained an object handle to the target process, it calls `VirtualAlloc` to allocate memory in the target process for injecting its shellcode. (Alternatively, it could call the `VirtualAllocEx` or `NtAllocateVirtualMemory` function.) The most relevant parameters for the `VirtualAlloc` function are `lpAddress` and `dwSize`, which represent the starting address and the size of the region of memory being allocated, respectively. The prototype information for the `VirtualAlloc` function is as follows:

```

VirtualAlloc(
    LPVOID lpAddress,      // Start address of region of memory to be allocated
  
```

```

SIZE_T dwSize,           // Size of memory allocation
DWORD flAllocationType, // Memory type to be allocated
DWORD flProtect          // Memory protection to be assigned to this region
);

```

After allocation, the malware writes the malicious code into this fresh memory region using `WriteProcessMemory` (or its native API equivalent, `NtWriteVirtualMemory`). `WriteProcessMemory` takes some important parameters: `hProcess`, the handle of the process being written into; `lpBaseAddress`, a pointer to the base address where the data will be written; `lpBuffer`, a pointer to a memory location that contains the data to be written; and `nSize`, the number of bytes to be written into the target process's memory. Here is the prototype information for the `WriteProcessMemory` function:

```

WriteProcessMemory(
    HANDLE hProcess,           // Handle of process being written to
    LPVOID lpBaseAddress,     // Base address where data will be written
    LPCVOID lpBuffer,         // Contains the data to be written
    SIZE_T nSize,             // Size of data to be written
    SIZE_T lpNumberOfBytesWritten // Optional parameter; pointer to a variable that receives
                                // the number of bytes that were written
);

```

Finally, after writing the malicious code into its target process, the malware is ready to execute it within the process context. To do so, it calls `CreateRemoteThread` (alternatively, `NtCreateThreadEx` OR `RtlCreateUserThread`) to create a remote thread within the process context and execute it. The most important parameters of `CreateRemoteThread` are `hProcess`, a handle to the target process, and `lpStartAddress`, the starting address of the code to be executed. The function prototype for `CreateRemoteThread` is as follows:

```

CreateRemoteThread(
    HANDLE hProcess,           // Handle to the process in which to create the
                                // thread
    LPSECURITY_ATTRIBUTES lpThreadAttributes, // Pointer to a security attributes structure;
                                                // pertains to security and access control
    SIZE_T dwStackSize,       // Initial size of the stack for the new thread
    LPTHREAD_START_ROUTINE lpStartAddress,    // Starting address of the new thread
    LPVOID lpParameter,       // Pointer to a variable that will be passed to
                                // the thread's function
    DWORD dwCreationFlags,     // Creation flags for the thread (such as
                                // CREATE_SUSPENDED)
    LPDWORD lpThreadId         // Pointer to a variable that receives the new
                                // Thread ID
);

```

Note that this is only one of the many methods of shellcode injection and execution. The flow of this technique and the functions being called

(OpenProcess, VirtualAlloc, WriteProcessMemory, and so on) are the basic building blocks of several techniques we’ll touch on in this chapter. Keep in mind as well that many of the functions listed in this chapter are interchangeable with other functions. For example, instead of WriteProcessMemory, malware might call the native API NtWriteVirtualMemory. Instead of CreateRemoteThread, it could invoke NtCreateThreadEx.

To trace process injection, I like to use API Monitor, which allows you to quickly see the way the malware is injecting code and even extract the injected code. You can also use it to inspect the parameters of each function call in an easy-to-read format.

NOTE

In the next section, I’ll be using a malware executable that you can download from VirusTotal or MalShare using the following file hash:

```
SHA256:
c39e675a899312f1e812d98038bb75b0c5159006e8df7a715f93f8b3ac23b625
```

Figure 12-2 shows the result of loading the malware sample in API Monitor and filtering on the OpenProcess, VirtualAlloc, and WriteProcessMemory functions.

Module	API	Return Value
explorer.exe	OpenProcess (STANDARD_RIGHTS_ALL PROCESS_CREATE_PROCESS PROCESS_CREATE_THREAD PROCESS_DUP_HANDLE PROCESS_QUERY_INFORMATION PROCESS_TERMINATE)	0x00000268
explorer.exe	VirtualAlloc (0x24010000, 401408, MEM_COMMIT MEM_RESERVE, PAGE_EXECUTE_READWRITE)	0x24010000
explorer.exe	VirtualAlloc (0x24020000, 401408, MEM_COMMIT MEM_RESERVE, PAGE_EXECUTE_READWRITE)	0x24020000
explorer.exe	VirtualAlloc (0x24030000, 401408, MEM_COMMIT MEM_RESERVE, PAGE_EXECUTE_READWRITE)	0x24030000
explorer.exe	VirtualAlloc (0x24040000, 401408, MEM_COMMIT MEM_RESERVE, PAGE_EXECUTE_READWRITE)	0x24040000
explorer.exe	VirtualAlloc (0x24060000, 401408, MEM_COMMIT MEM_RESERVE, PAGE_EXECUTE_READWRITE)	0x24060000
explorer.exe	VirtualAlloc (0x24070000, 401408, MEM_COMMIT MEM_RESERVE, PAGE_EXECUTE_READWRITE)	0x24070000
explorer.exe	VirtualAlloc (0x240a0000, 401408, MEM_COMMIT MEM_RESERVE, PAGE_EXECUTE_READWRITE)	0x240a0000
explorer.exe	VirtualAlloc (0x240e0000, 401408, MEM_COMMIT MEM_RESERVE, PAGE_EXECUTE_READWRITE)	0x240e0000
explorer.exe	VirtualAlloc (0x240f0000, 401408, MEM_COMMIT MEM_RESERVE, PAGE_EXECUTE_READWRITE)	0x240f0000
explorer.exe	WriteProcessMemory (0x00000268, 0x07540000, 0x0072c910, 13, 0x0019fd78)	TRUE
explorer.exe	WriteProcessMemory (0x00000268, 0x07550000, 0x0019fd98, 12, 0x0019fd44)	TRUE
explorer.exe	WriteProcessMemory (0x00000268, 0x09110000, 0x00403920, 210, 0x0019fd44)	TRUE
explorer.exe	CreateRemoteThread (0x00000268, NULL, 0, 0x09110000, 0x07550000, 0, 0x0019fd68)	NULL

Figure 12-2: Shellcode injection captured in API Monitor

As you can see, this sample begins by calling OpenProcess (with the STANDARD_RIGHTS_ALL permission and several other access permissions), then makes multiple calls to VirtualAlloc to allocate memory in the target process. The malware then writes code into that process using WriteProcessMemory and executes it by invoking CreateRemoteThread.

If you inspect the buffer for the third call to WriteProcessMemory, you can see that the malware is writing what looks like shellcode into the target process (see Figure 12-3).

NOTE

It takes some practice, but you can identify shellcode by inspecting the data and looking for bytes that represent common assembly instructions such as

8b ec (which translates to `mov ebp, esp`). Refer back to [Chapter 3](#) for additional assembly instructions.

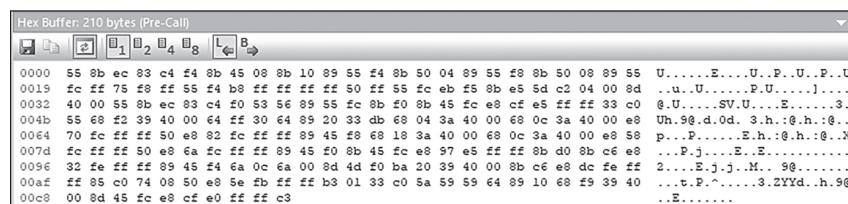


Figure 12-3: Shellcode injected into a target process

To verify that it's shellcode, save the data (click the Save icon in the upper-left corner of the hex buffer window) and then view it in a disassembler. You should see the following:

```
--snip--
push ebp
mov  ebp, esp
add  esp, 0FFFFFFF4h
mov  eax, [ebp+8]
mov  edx, [eax]
mov  [ebp-0Ch], edx
mov  edx, [eax+4]
mov  [ebp-8], edx
mov  edx, [eax+8]
mov  [ebp-4], edx
push dword ptr [ebp-8]
call dword ptr [ebp-0Ch]
--snip--
```

Because this data cleanly converts to assembly code, this indeed appears to be shellcode. I won't go into this code in more detail here, but if I were analyzing this malware sample, I would try to understand this code's purpose by investigating it further in the disassembler and then investigating it dynamically in a debugger. Note that in some circumstances, the disassembler may incorrectly recognize this as data instead of code. (Refer back to the "Disassembly" box on [page 51](#) for a refresher on the code vs. data problem.) You might have to "force" the disassembler to read it as code.

Before we move on, there's something else injected into the target process worth noting. If you inspect the buffer for the first call to

WriteProcessMemory, you'll see a reference to *kernel32.dll* (see [Figure 12-4](#)).



Figure 12-4: The string *KERNEL32.DLL* written into memory

This indicates that the sample could also be using another process injection technique, DLL injection, which we'll take a look at now.

DLL Injection

While DLL injection is another common form of process injection, don't be misled by its name. In this type of attack, the malware doesn't physically inject a DLL into a target process; instead, it writes *the path* to a malicious DLL file located on disk into the target process and then forces the target process to load and execute that DLL on its behalf. **Figure 12-5** illustrates this technique.

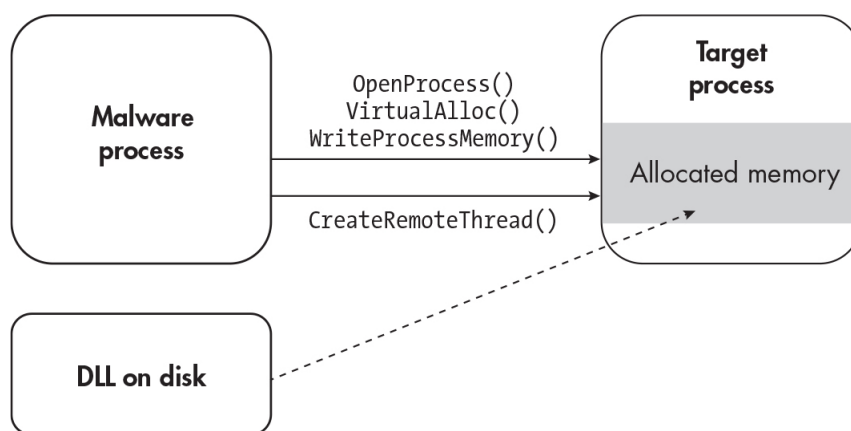


Figure 12-5: The DLL injection technique

The malware first drops a malicious DLL file to disk. Then, as in shellcode injection, it invokes `OpenProcess` to get a handle to its chosen target process and uses `VirtualAlloc` to allocate memory within that process. Next, it calls `WriteProcessMemory` to write the location of that DLL file into the process. Finally, to make the victim process load its DLL, the malware gets the procedural address of `LoadLibrary` and then calls `CreateRemoteThread` with the `lpStartAddress` parameter set to that address. Once the remote thread executes, the target process calls `LoadLibrary` and, in doing so, loads the malicious DLL. Here's how this looks in pseudocode:

```

WriteProcessMemory(victimProcess, lpBaseAddress, maliciousDllName, nSize,
lpNumberOfBytesWritten);
hModule = GetModuleHandle("Kernel32.dll");
GetProcAddress(hModule, "LoadLibraryA");
CreateRemoteThread(victimProcess, lpThreadAttributes, dwStackSize,
addressOfLoadLibraryA, maliciousDllName, dwCreationFlags, lpThreadId);
  
```

This malware calls `WriteProcessMemory` to write the malicious DLL path, `maliciousDllName`, into the target process, `victimProcess`. Then, the malware calls `GetModuleHandle`, followed by `GetProcAddress`, to get the procedural address of the `LoadLibraryA` function. Finally, the malware calls `CreateRemoteThread`, passing the address of `LoadLibraryA` and the path to the malicious DLL as

parameters. This forces the victim process to load the malicious DLL and execute the code within a new thread.

One problem with traditional DLL injection for malware authors is that the DLL must be loaded from disk, using the standard Windows library loading procedures. These standard loading procedures are monitored and easily spotted by endpoint defenses. A stealthier approach to DLL injection is reflective DLL injection.

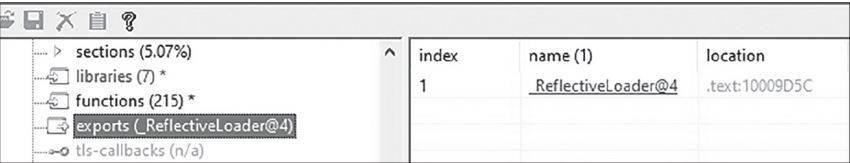
Reflective DLL Injection

In *reflective* DLL injection, the DLL is stored in memory rather than on disk, and the malware loads it without having to rely on the standard Windows loading mechanism. This makes reflective DLL injection a more covert alternative to the standard DLL injection method just described.

The initial steps of reflective DLL injection are fairly similar to the standard DLL injection. The malware obtains a handle to its victim process using `OpenProcess` and allocates memory in that process using `VirtualAlloc`. However, instead of writing only the path to the DLL file, the malware copies the entire malicious DLL to the target process’s memory. Then, it transfers control flow to the newly injected DLL (using `CreateRemoteThread`, for example), which executes the injected DLL’s “bootstrap” loader code.

This bootstrap code is custom code that must re-create the normal Windows DLL loading process. At a high level, these are the steps:

- 1. The bootstrap code calculates its own image location in memory and performs its own image base relocations, which means realigning the hardcoded addresses in the executable code to match its current location in memory. The loader also finds the location of the injected DLL’s PEB.
- 2. The bootstrap code parses the exports table of *kernel32.dll* to locate the addresses of `LoadLibrary`, `GetProcAddress`, `VirtualAlloc`, and other fundamental functions.
- 3. The malicious DLL has now been successfully loaded inside the victim process and is ready to run. To execute the DLL’s malicious code, malware typically invokes a function in the DLL’s exports table, such as the function shown in **Figure 12-6**.



index	name (1)	location
1	ReflectiveLoader@4	.text:10009D5C

Figure 12-6: The DLL export function

The malware shown here was generated with the popular penetration testing tool Cobalt Strike and has `ReflectiveLoader@4` as the default export function. Once this export function is invoked, the malicious code is executed.

Figure 12-7 illustrates a typical reflective DLL injection attack.

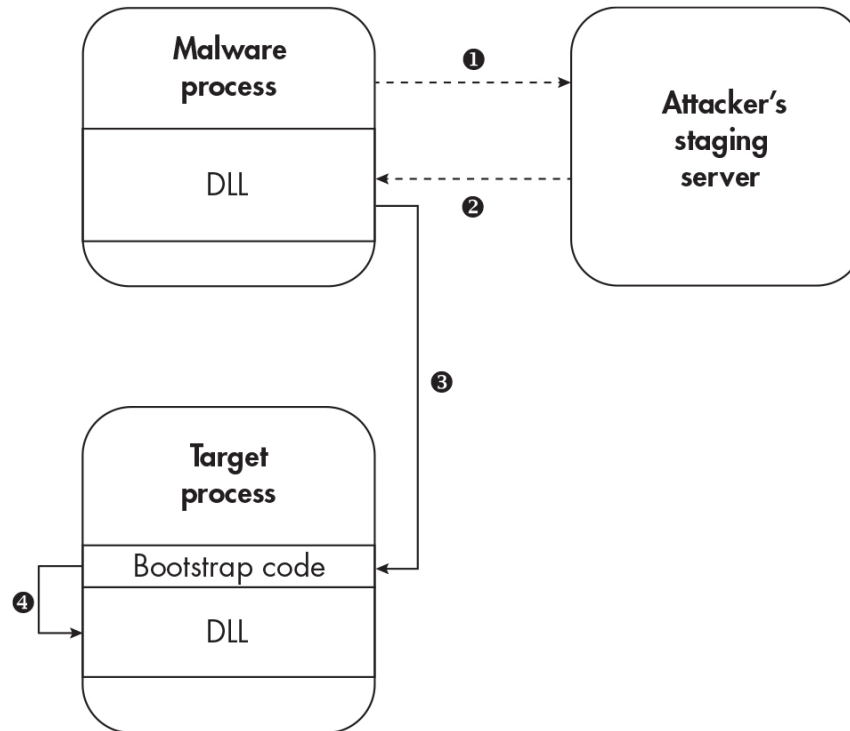


Figure 12-7: The reflective DLL injection technique

In this figure, the running malware downloads its DLL payload from a staging server controlled by the attacker ①, which is then temporarily stored in the malware's process memory space ②. (Note that this technique does not always rely on a remote staging server, but we'll touch on that in a moment.) Next, the bootstrap code is injected into a target process along with the DLL ③, then executed by the malware. The bootstrap code performs the manual DLL loading process and then calls the export function of the injected DLL, which executes the malicious code in the context of the target process ④. You can read more about reflective DLL injection from the original author of the technique at <https://github.com/stephenfewer/ReflectiveDLLInjection>.

Reflective DLL injection, as with many of the injection methods I discuss throughout this chapter, can be subdivided into so-called staged and stageless techniques. The technique just presented is considered *staged*, as the payload to be injected is hosted and downloaded from the attacker's staging server. In *stageless* reflective DLL injection, the payload is already embedded in the original malware executable before being unpacked and injected into the target process.

A similar technique, sometimes called *shellcode reflective injection*, involves converting a DLL into shellcode and then injecting it into a target process. I won't cover this technique further since it combines techniques you've already seen, but you can read more about it at <https://github.com/monoxgas/sRDI>.

Process Hollowing

Process hollowing (sometimes called *RunPE*, *process replacement*, or *hollowed process injection*) involves unmapping code from a target process's memory and then remapping malicious code there. Process hollowing is a bit different from the other injection techniques we've seen so far, as it doesn't usually involve an arbitrary remote process. Instead, the malware starts a new process (often a trusted executable, such as our beloved *Calculator.exe*), unmaps the legitimate code, and remaps malicious code in a suspended state, at which point the malware executes the malicious code. This hides the malicious code from the prying eyes of endpoint defenses and investigators, disguising it as a normal process. **Figure 12-8** illustrates the technique.

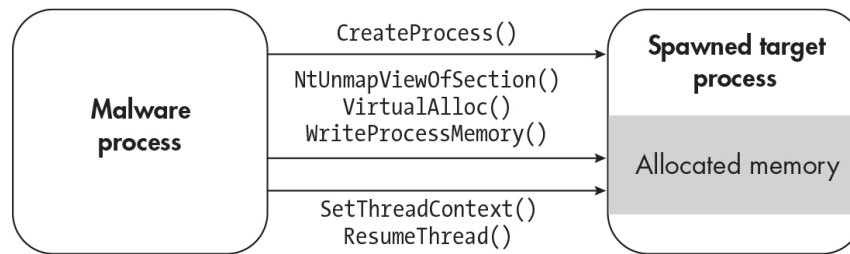


Figure 12-8: The process-hollowing technique

This malware sample begins by creating a new process with `CreateProcess` and then sets the process creation flag to `CREATE_SUSPENDED` (not shown); this starts the new process in a suspended state. Next, the malware “hollows out” the newly created process by unmapping its legitimate code with the function `NtUnmapViewOfSection`. Then, the malware allocates a new region of memory for its malicious code using `VirtualAlloc` and writes (maps) its payload here using `WriteProcessMemory`. Finally, the malware calls `SetThreadContext` and `ResumeThread` to point the current thread to the newly injected code and resume thread execution, respectively, which runs the malicious code.

Let's take a look at this in practice, using a variant of the ransomware family Satan (SHA256: `cbbd2bd6f98bd819f3860dea78f812d5c180fd19924cef32e94Bd7f6929023779`). The screen capture from API Monitor in **Figure 12-9** shows a malware sample (a variant of the Satan ransomware family) using the process-hollowing technique.

API
CreateProcessW (NULL, ""C:\Users\Public\financialz.exe", NULL, NULL, FALSE, CREATE_SUSPENDED, NULL, NULL, 0x0019f8e8, 0x0019f950)
NtReadVirtualMemory (0x0000022c, 0x00241008, 0x0019f948, 4, NULL)
NtUnmapViewOfSection (0x0000022c, 0x00400000)
VirtualAllocEx (0x0000022c, 0x00400000, 73728, MEM_COMMIT MEM_RESERVE, PAGE_EXECUTE_READWRITE)
NtWriteVirtualMemory (0x0000022c, 0x00400000, 0x007d2cf0, 1024, NULL)
NtWriteVirtualMemory (0x0000022c, 0x00401000, 0x007d30f0, 35328, NULL)
NtWriteVirtualMemory (0x0000022c, 0x0040a000, 0x007dbaf0, 17408, NULL)
NtWriteVirtualMemory (0x0000022c, 0x0040f000, 0x007dfef0, 1024, NULL)
NtWriteVirtualMemory (0x0000022c, 0x00411000, 0x007e02f0, 2560, NULL)
NtWriteVirtualMemory (0x0000022c, 0x00241008, 0x007d2df4, 4, NULL)
NtSetContextThread (0x00000228, 0x0019f990)
NtResumeThread (0x00000228, NULL)

Figure 12-9: A malware sample using the process-hollowing technique

Notice how this malware starts a new process with `CreateProcessW` from an executable on disk (*financialz.exe*) in a suspended state (`CREATE_SUSPENDED`), as just discussed. Then, it makes multiple calls to `NtWriteVirtualMemory` to map data into the new target process. Inspecting this function call further in API Monitor reveals that this malware sample is writing a PE file into the target process, then calling `NtResumeThread` to execute it (see [Figure 12-10](#)).

Hex Buffer: 1024 bytes (Pre-Call)	
0000 4d 5a 90 00 03 00 00 00 04 00 00 00 ff ff 00 00 b8 00 00 00	MZ.....
0014 00 00 00 00 40 00 00 00 00 00 00 00 00 00 00 00 00 00 00	@.....
0028 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
003c d0 00 00 00 0e 1f ba 0e 00 b4 09 cd 21 b8 01 4c cd 21 54 68	!..L!Th
0050 69 73 20 70 72 6f 67 72 61 6d 20 63 61 6e 6e 6f 74 20 62 65	is program cannot be
0064 20 72 75 6e 20 69 6e 20 44 4f 53 20 6d 6f 64 65 2e 0d 0d 0a	run in DOS mode....
0078 24 00 00 00 00 00 00 00 00 22 68 35 e1 66 09 5b b2 66 09 5b b2	\$....."hs.f.[.f.[.
008c 66 09 5b b2 66 09 5b b2 67 09 5b b2 6f 71 c9 b2 75 09 5b b2	f.[.f.[.g.[.eq.u.[.
00a0 66 09 5a b2 ff 09 5b b2 6b 5b be b2 7d 09 5b b2 6b 5b 85 b2	f.z...[.k[...].[k[...
00b4 67 09 5b b2 52 69 63 68 66 09 5b b2 00 00 00 00 00 00 00 00	g.[.Richf.[.....
00c8 00 00 00 00 00 00 00 00 00 50 45 00 00 4c 01 04 00 f8 20 89 58	DE T. X

Figure 12-10: The PE header in memory

Note that while process hollowing is typically classified as a process injection technique, it's not a true injection technique, since it relies on the malware spawning a new process in a suspended state and then replacing the process's preexisting code with malicious code.

Thread Hijacking

Thread hijacking involves opening a running thread in a victim process, writing malicious code into that thread, and forcing the victim process to execute the code. Thread hijacking shares many of the functions used in the process-hollowing technique, but there are a few notable differences. To execute thread hijacking, the malware invokes the `OpenThread` function, specifying a thread ID as a parameter, and then calls `SuspendThread` (or `Wow64SuspendThread` for 64-bit processes) to suspend the victim thread. Once the thread is in a suspended state, the malware uses `VirtualAlloc` and `WriteProcessMemory`, respectively, to allocate memory in the target process and write its malicious code there. Finally, the malware calls `SetThreadContext` to transfer control flow from the currently suspended thread to the newly injected malicious code, then calls `ResumeThread` to execute the code.

APC Injection

An *asynchronous procedure call (APC)* is a Windows feature that allows for various tasks to be queued and executed in the context of a running thread. A program might invoke the `QueueUserAPC` function, passing both the handle to the thread and a pointer to the code that the program wishes to run, to add that task to the APC queue. *APC injection* abuses this functionality to stealthily execute code and, potentially, elevate privileges.

For a program to be able to call functions in the APC queue, the thread must be in an *alertable* state, meaning the thread periodically checks for new items in the queue and runs the next queued task. Many processes running on the system, from web browsers to video players, have threads running in alertable states. Generally, such threads eventually receive an interrupt request from the operating system, at which point the process will inspect the APC queue and run the next queued task.

Malware takes advantage of APC functionality by attempting to inject malicious code into other processes via APC queuing. [Figure 12-11](#) illustrates this attack.

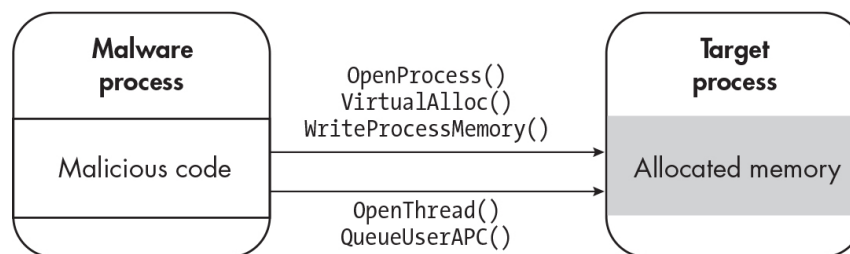


Figure 12-11: The APC injection technique

This malware first gets a handle to its target victim process in the usual way (`OpenProcess`) and writes the malicious code to be executed into the process (`VirtualAlloc` and `WriteProcessMemory`). Next, it opens a target thread by calling `OpenThread`, followed by `QueueUserAPC` (or `NtQueueApcThread`) to queue a new APC task. Whenever the thread receives an interrupt request from the operating system, this malicious code will be executed.

A variation of this technique creates a new process in a suspended state (similar to process hollowing) and writes the malicious code into it. Then, the malware queues its APC function and resumes the thread. Once the thread resumes, the malicious code will be executed. At this point, the malware can terminate its original process, as its payload is now running under a new process and may go unnoticed by endpoint defenses and unaware end users. [Figure 12-12](#) shows this attack, captured in API Monitor.

Module	API
wollez.exe	CreateProcessA (NULL, "C:\Program Files\Internet Explorer\iexplore.exe", NULL, NULL, FALSE, CREATE_SUSPENDED, NULL,
wollez.exe	WriteProcessMemory (0x0000000000000174, 0x000001a8b2da0000, 0x00007ff74a6a5070, 261641, NULL)
wollez.exe	QueueUserAPC (0x000001a8b2da0000, 0x00000000000000a8, 0)
KERNELBASE.dll	↳ NtQueueApcThread (0x00000000000000a8, 0x00007ffe7b69c940, 0x000001a8b2da0000, NULL, 0)
wollez.exe	ResumeThread (0x00000000000000a8)
KERNELBASE.dll	↳ NtResumeThread (0x00000000000000a8, 0x000000a9ea8ff648)

Figure 12-12: Malware using a variation of the APC injection technique

This malware sample uses APC injection to run malicious code in the context of the *iexplore.exe* process. In this case, the malware starts *iexplore.exe* (`CreateProcessA`) instead of hijacking another process (`OpenProcess`).

Atom Bombing

The last process injection technique we'll cover in this chapter is *atom bombing*, which shares some features of APC injection but involves *atoms*, references to specific pieces of data such as strings. Atoms are stored in an OS structure known as the *atom table*, and each atom has a unique atom identifier. Atoms are often used in interprocess communication to coordinate actions between processes. For example, Process A might create an atom to indicate that a particular piece of data is available, and Process B can use that atom to access the data.

Atom tables can be global (that is, accessible from any process running on the system) or local (accessible to only one specific process). To add data to the global atom table, an application invokes the `GlobalAddAtom` function; to add an atom to a local table, it calls the `AddAtom` function. Atom bombing abuses these atom tables to temporarily store malicious code. **Figure 12-13** illustrates this attack.

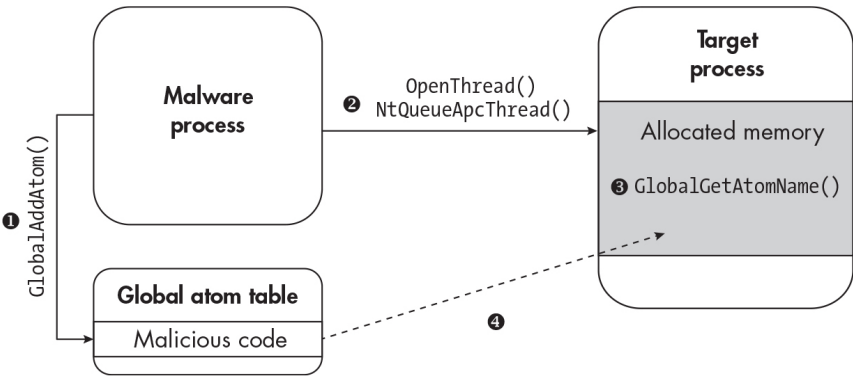


Figure 12-13: The atom bombing technique

At a high level, this technique creates a new global atom (`GlobalAddAtom`) in the global atom table, where it stores malicious shellcode ❶ and uses APC queuing (`NtQueueApcThread`) ❷ to force the victim process into executing `GlobalGetAtomName` ❸. Once the atom is retrieved by the victim process, the shellcode stored in the atom table can be run in that process's context ❹.

Process Injection Wrap-up

As you've seen, many process injection techniques use the same API function calls and behave similarly. Nearly all process injection techniques create a process or open a handle to a target process, allocate memory and write or map code into the target process, and then force the injected code to execute in the context of the target process, as summarized in **Figure 12-14**. (Special thanks to malware researcher Karsten Hahn, who inspired this image.)

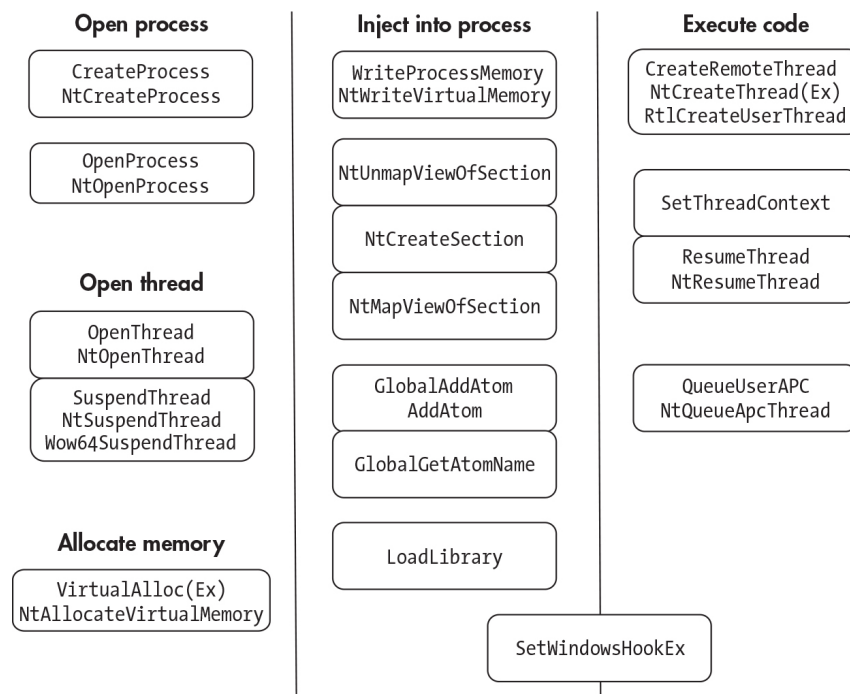


Figure 12-14: A summary of process injection behaviors and API calls

Now that we've looked at various process injection techniques, let's pivot to another method used for similar purposes: process image manipulation.

Process Image Manipulation

Process image manipulation abuses both the standard Windows process creation routine and the way endpoint defenses (specifically, anti-malware software) operate. To defend a system from malware attacks, anti-malware software needs to know when a new process is started in order to monitor it for suspect behaviors. This is where the API function `PsSetCreateProcessNotifyRoutineEx` comes in. When a new process is created, this function sends notification messages to endpoint defense software, which then inspects and scans the originating executable file that started the process. If the anti-malware software sees that the executable file contains malicious code, it can quarantine the file and terminate any associated processes. The problem is that `PsSetCreateProcessNotifyRoutineEx` is in-

voked not at the exact moment of a process's creation, but rather on the creation and execution of the first few threads running within the process context. This gives malware a nice window of opportunity to modify the original executable file before anti-malware solutions have time to scan it.

Process manipulation also involves interfering with the way Windows creates processes (as described in [Chapter 1](#)). In essence, Windows usually calls a function such as `NtCreateUserProcess`, a kernel function that handles the details of process creation and then maps processes into memory by executing the following steps:

1. Obtain a handle to an executable file (by calling `CreateFile`, for example).
2. Create a section object for the file, typically by calling the `NtCreateSection` function. In this case, a *section* is simply an object that will be mapped into memory.
3. Create a process object by calling `NtCreateProcessEx` with a parameter referencing the newly created section object, mapping the process object into memory as a new process.
4. Execute the process by calling `NtCreateThreadEx` to create and start a new thread.

Let's take a look at how different process manipulation techniques are used to interfere with these steps.

Process Herpaderping

Process herpaderping has a funny name but packs a powerful punch. It confuses the operating system and anti-malware solutions by interfering with the way Windows creates processes. [Figure 12-15](#) illustrates how this technique works.

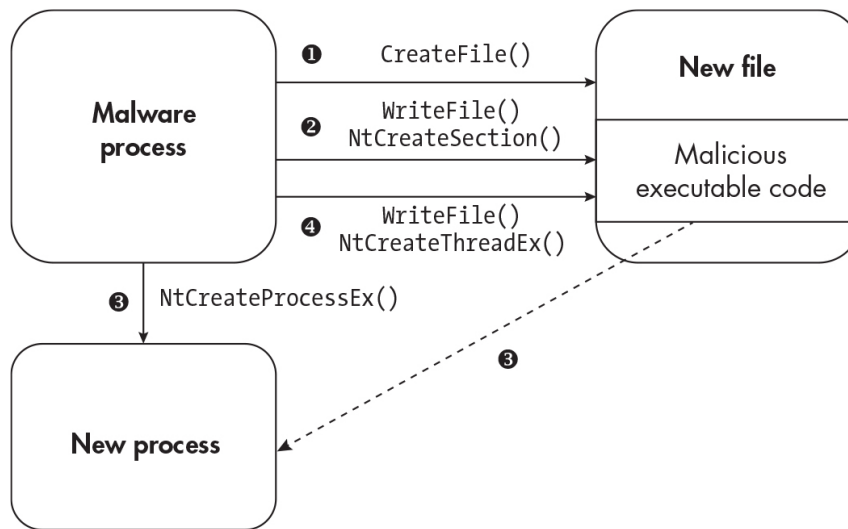


Figure 12-15: The process-herpaderping technique

This malware first creates a new file on disk and obtains a handle to it, keeping that handle open ❶. It writes malicious executable code into this empty file, then creates a section object for the file ❷. Next, it creates a process using the new section object ❸. The malicious code is now mapped into memory, but it isn't executing yet. So far, this approach follows the standard Windows PE loading steps, but things are about to get interesting.

The malware modifies or removes the malicious code within the file it created on disk ❹. It then starts a new thread (`NtCreateThreadEx`), which executes the malicious code in memory, and closes the handle to the open file. At this point, the aforementioned `PsSetCreateProcessNotifyRoutineEx` callback will occur and endpoint defenses will kick in, inspecting the malicious file. However, since the malicious file in fact no longer contains malicious code, the anti-malware software is tricked into thinking all is well. Anti-malware software, as well as Windows itself, assumes that a file on disk can't (or shouldn't) be modified when it's already mapped into memory and running inside a process. Process herpaderping takes advantage of such assumptions to execute malicious code. You can find out more about this technique from its author at <https://jxy-s.github.io/herpaderping/>. If you'd like to test it yourself, you can explore the proof of concept at <https://github.com/jxy-s/herpaderping>.

Process Doppelganging

Process doppelganging, first presented at Black Hat 2017 by security researchers Eugene Kogan and Tal Liberman, is a manipulation technique that uses Transactional NTFS to hide malicious code execution. The term *doppelganging* originates from the word *doppelganger*, which is commonly used to describe a person with an uncanny resemblance to someone else. *Transactional NTFS* was designed to add extra features and sup-

port (like file integrity preservation and better error handling) to NTFS, the default filesystem in Windows. It allows the use of *transactions*, which track changes to the filesystem that can be rolled back when necessary. Actions like file deletion first occur virtually. If the file deletion request succeeds, the transaction is committed and the file is actually deleted. If the file deletion request results in an error, however, the transaction is rolled back and no action occurs. Ultimately, transactions seek to prevent data inconsistency and corruption resulting from a system failure or other unexpected event.

A new file created with transactions normally can't be accessed from outside the process currently interacting with it; even certain anti-malware solutions can't access it in this case. For this reason, transactions can provide a safe location for a malicious file to temporarily hide. And since transacted files can be "rolled back" to a previous state, endpoint defenses can easily become confused. This is exactly why process doppelganging is effective. **Figure 12-16** illustrates how this technique works.

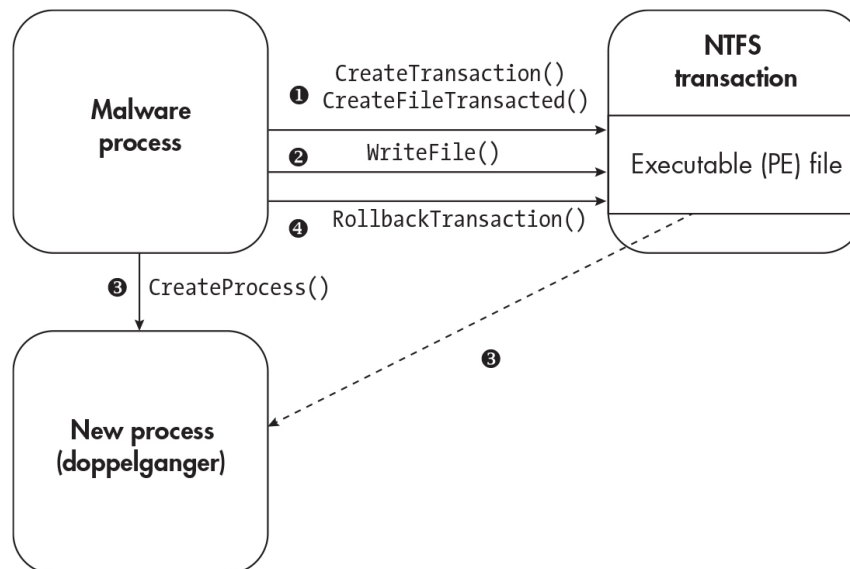


Figure 12-16: The process-doppelganging technique

This malware creates a new NTFS transaction using `CreateTransaction` and opens an existing executable file using `CreateFileTransacted` ❶. (It could instead call lower-level functions such as `ZwCreateTransaction`, `RtlSetCurrentTransaction`, and `ZwCreateFile` to achieve the same effect.) Next, the malware writes its malicious code into the existing PE file (`WriteFile`), which will replace the original executable's code ❷. The malware then executes `CreateProcess` or `NtCreateProcessEx` to create a new process, passing the location of the new malicious PE file (the doppelganger) as a parameter ❸. Once the PE file is mapped into memory and the malicious process is running, the malware deliberately rolls back the original NTFS transaction (`RollbackTransaction`), returning the file to its original, preinfected state ❹. When the anti-malware software is notified of the process creation

and scans the process's executable file on disk, it's too late; the original executable file has already been rolled back to its nonmalicious state.

A few variations of the process-doppelganging technique have emerged since it was first published in 2017. One variant, dubbed *transacted hollowing* by researchers at Malwarebytes, combines process hollowing with process doppelganging (see the blog post "Process Doppelganging Meets Process Hollowing in Osiris Dropper" at <https://www.malwarebytes.com/blog/news/2018/08/process-doppelganging-meets-process-hollowing-osiris>). As with doppelganging, the malware creates a new process for the injected payload. However, instead of directly injecting the payload, it creates the process in a suspended state and maps the malicious code into memory, as in process hollowing. This is a good example of how security research is often used later by malware authors as a jumping-off point for new techniques.

As you've seen, process image manipulation techniques rely on the exploitation of fundamental Windows behaviors, and they're unlikely to go away anytime soon. It's important for analysts to keep current on what's cropping up in the wild. To close out this section, we'll look at two newer process manipulation techniques.

Process Reimaging and Ghosting

Process reimaging is similar to the previously described techniques in that it relies on manipulating a currently running process to circumvent security controls. To execute process reimaging, the malware modifies its `FILE_OBJECT` attribute, which contains the filepath of the malware's executable on disk, to point instead to a benign, legitimate executable. This technique relies on the fact that there are inconsistencies in the way Windows handles `FILE_OBJECT`, and as a result, some anti-malware products put too much trust in what's stored in a malicious process's `FILE_OBJECT` location. More information on this technique can be found in the blog post "In NTDLL I Trust" from McAfee (<https://www.mcafee.com/blogs/other-blogs/mcafee-labs/in-ntdll-i-trust-process-reimaging-and-endpoint-security-solution-bypass/>) and on GitHub (<https://github.com/djhohnstein/ProcessReimaging>).

Process ghosting is most similar to process herpaderping. The malware creates a file and requests that Windows puts the file into a pending deletion state. Since there can be a delay between when the file is put into this state and when it's actually deleted, the malware can write malicious executable code into the file and create an image object for it (copying the file content into memory) before Windows deletes it. Finally, the malware creates a process with the image object of the now-deleted file and executes it. Since Windows prevents endpoint defenses like anti-malware

from reading and inspecting the file while it's in a pending deletion state, these defenses are effectively blinded to the malicious code in the file. You can read more about this technique in the blog post "What You Need to Know About Process Ghosting" at <https://www.elastic.co/blog/process-ghosting-a-new-executable-image-tampering-attack>.

Now we'll turn to another pair of methods for injecting and covertly executing code.

DLL and Shim Hijacking

Hijacking is a general term for techniques that involve interfering with the normal execution flow of a program or manipulating the way Windows runs programs, in order to execute unauthorized code. Specifically, *DLL hijacking* exploits the way legitimate executables load their required libraries, in order to inject a malicious DLL. *Shim hijacking* involves using Windows application *shims*, or small libraries that intercept API calls, to inject code into an executing process. Let's dig a little deeper into each technique.

DLL Hijacking

All Windows applications must load DLL files at some point to function. Most applications have some sort of manifest that lists all the required DLLs and where they reside on disk. This manifest is part of the application's *DLL search order*. Other possible search order locations include the application's default install directory (such as *C:\Program Files\CoolApplication*), where its primary executable file is located, and Windows directories that house the standard Windows DLLs (such as *%SystemRoot%\system32*).

This setup leaves open an interesting attack vector. Many poorly written applications don't verify the contents or signature of the DLLs they're loading; instead, they blindly load the required files based on their search order. Worse still, some poorly written applications simply load all DLLs in their install or running directory haphazardly. If a threat actor knows that a particular application is vulnerable to DLL hijacking, it could craft a specialized piece of malware that drops its payload in the form of a DLL to the directory from which the application is loading its DLLs. The actor can then simply wait for the application to start and autoload the DLL into memory, or it can execute the application directly and force the malicious DLL to load. Lastly, some malware will directly modify an application's DLL search order or manifest to force a malicious DLL to load. There are several variations of this attack (DLL search order hijacking, sideloading, preloading, remote preloading, and so on), but they all have the same effect: deviously and silently loading a malicious DLL into

the vulnerable application's process memory space and executing unauthorized code.

An example of this attack comes from the Qbot malware family. One Qbot variant was delivered to a victim with a few files, most notably a malicious DLL file (*WindowsCodecs.dll*), a copy of an older but legitimate version of Windows Calculator (*calc.exe*), and a DLL called *7533.dll*. The *calc.exe* file had a curious import in its IAT, highlighted in [Figure 12-17](#).

sections (file)	comctl32.dll	0x2104	0x51E24	implicit	9	Common Controls Library
libraries (97)	ntdll.dll	0x112C	0x51E4C	implicit	5	NT Layer DLL
imports (381) *	kernel32.dll	0x1144	0x51E64	implicit	88	Windows NT BASE API Client DLL
exports (n/a)	user32.dll	0x12A8	0x51FC8	implicit	97	Multi-User Windows USER API Client DLL
exceptions (n/a)	rpcrt4.dll	0x4430	0x52150	implicit	3	Remote Procedure Call Runtime
→ file callbacks (n/a)	wsxmem.dll	0x4440	0x52160	implicit	1	MCJ API DLL
relocations (746)	version.dll	0x4448	0x52168	implicit	3	Version Checking and File Installation Libraries
resources (91) *	gdi32.dll	0x4458	0x52178	implicit	27	GDI Client DLL
exe strings (size)	msvcrt.dll	0x44C8	0x521E8	implicit	89	Windows NT CRT DLL
debug (path)	WindowsCodecs.dll	0xFFFFFFFF	0xFFFFFFFF	delay-load	1	Microsoft Windows Codecs Library
manifest (advoker)						

Figure 12-17: A curious import in the *calc.exe* file delivered from a Qbot variant

Upon execution, starting with its present location, this *calc.exe* application searches for *WindowsCodecs.dll*, the real version of which is a legitimate, benign Windows helper application. Since the malware author “helpfully” included *WindowsCodecs.dll* alongside the *calc.exe* file, the malicious version of the DLL is injected into *calc.exe* and its code is executed. [Figure 12-18](#) illustrates this attack in Procmon.

Process Name	PID	Operation	Path	Result	Detail
calc.exe	3232	Process Start		SUCCESS	Parent PID: 4236, Command line: "C:\Users\Public\calc.exe"
calc.exe	3232	Thread Create		SUCCESS	Thread ID: 5288
calc.exe	3232	Thread Create		SUCCESS	Thread ID: 4792
calc.exe	3232	Load Image	C:\Users\Public\WindowsCodecs.dll	SUCCESS	Image Base: 0x79400000, Image Size: 0x6000
calc.exe	3232	Process Create	C:\Windows\SysWOW64\regsvr32.exe	SUCCESS	PID: 2552, Command line: C:\Windows\SysWOW64\regsvr32.exe 7533.dll

Figure 12-18: The malicious *WindowsCodecs.dll* file being sideloaded

This Procmon timeline export illustrates the victim executing the vulnerable *calc.exe* file (Process Start), which then loads the malicious *WindowsCodecs.dll* file (Load Image), which in turn executes another payload (*regsvr32.exe 7533.dll*). The malware author knows that this particular older version of *calc.exe* is vulnerable to DLL hijacking because it blindly loads and executes any DLL named *WindowsCodecs.dll* in its running location. Very crafty indeed!

Shim Hijacking

Microsoft's Application Compatibility framework allows for application *shimming*, or adding compatibility for software designed for older versions of Windows so that it can run on more recent versions of the OS. Developers can use shims to apply patches to a program without having to rewrite or recompile code. Shims are not only great tools for developers, though; using them is also a powerful method of code injection for malware. Using shims, malware can intercept API calls and modify their parameters. When a user starts an application, Windows starts the shim engine and checks to see if that application has any installed shims. Malware can take advantage of this behavior by abusing the shim

InjectDLL, which, as its name implies, injects a DLL module into the shimmed application. Once the application is started via its executable file, a malicious DLL file is also loaded into the image of the victim application and executed.

A malicious actor can create a shim on the victim system by using the built-in Windows shim database installer tool *sdbinst.exe*. The malware invokes this tool and points it to the malicious shim database (.sdb) file like so:

```
C:\> sdbinst.exe firefox.sdb
```

Once the malware installs a shim on a victim host, the shim database is installed in *C:\Windows\AppPatch\Custom* or *C:\Windows\AppPatch\AppPatch64\Custom\Custom64* (for 64-bit applications) in the form of one or more .sdb files. **Figure 12-19** shows what such a file might look like.

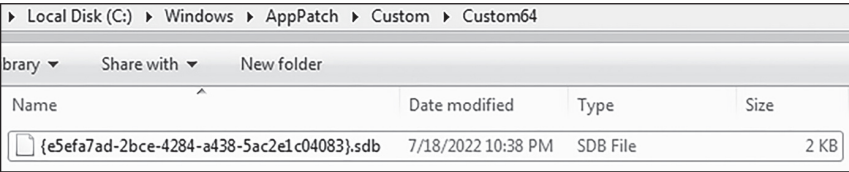


Figure 12-19: An .sdb file installed on a Windows system

Installed shims typically have a registry entry in either *HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\AppCompatFlags\Custom* or *HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\AppCompatFlags\InstalledSDB*, as shown in **Figure 12-20**.

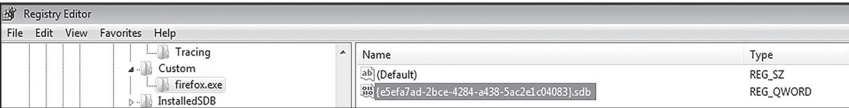


Figure 12-20: Shim database registry entries

During an investigation, you can explore suspicious .sdb files in more detail by using the aptly named SDB Explorer tool, available for free from <https://ericzimmerman.github.io/> and shown in **Figure 12-21**.

Name	Value
File name	C:\Users\Public\8b0dd304619374cf30ca74979f9371d5250dadb14107...
INDEXES	
DATABASE	
TIME	5/19/2014 11:28:47 AM +00:00
COMPILER_VERSION	2.1.0.3
NAME	Apps32
OS_PLATFORM_OR_...	1
DATABASE_ID	8a4d5a43-c64a-45ab-bdf4-804fe18ceafd
LIBRARY	
LAYER: SPVC32ldr	
EXE: chrome.exe	
EXE: explorer.exe	
EXE: firefox.exe	
EXE: iexplore.exe	
STRINGTABLE	
STRINGTABLE_ITEM	2.1.0.3
STRINGTABLE_ITEM	Apps32
STRINGTABLE_ITEM	SPVC32ldr
STRINGTABLE_ITEM	InjectDll
STRINGTABLE_ITEM	\\.\globalroot\systemroot\apppatch\spbin\spvc32loader.dll

Figure 12-21: Analyzing an .sdb file in SDB Explorer

As you can see in this output, the file seems to be shimming *chrome.exe*, *explorer.exe*, *firefox.exe*, and *iexplore.exe*. You can also tell that it's using the *InjectDll* functionality and referencing a DLL file (*spvc32loader.dll*). Based on this information, we can suspect that this shim is targeting browser processes and attempting to inject a malicious DLL into their respective memory address space, which will occur when the browser process first executes.

A good example of shim hijacking is documented in a report from Mandiant on the threat group FIN7, which was able to install the Carbanak backdoor and persist on the infected endpoints by registering a new shim database and patching the legitimate Windows *services.exe* executable. *Services.exe* is a critical system process and always runs when Windows first boots up. Once *services.exe* executes, the shim executes and launches the malicious Carbanak payload. For full details on the attack, see <https://www.mandiant.com/resources/fin7-shim-databases-persistence>.

Shims are a multipurpose evasive mechanism. They're a means of covertly injecting code to fly under the radar of endpoint defenses and investigators, as well as a way to establish persistence (as demonstrated by the FIN7 attack). In addition to these purposes, shims can serve as a rudimentary method for hooking. Let's look at how hooking works.

Hooking

As described in [Chapter 8](#), hooking is a technique used to intercept, monitor, and possibly modify Windows function calls. It has both malicious and legitimate applications; for example, hooking can be used by the following applications:

- Benign applications (such as shims), to modify or patch code or to monitor a system or other applications
- Sandboxes and other malware analysis tools, to monitor the malware's behaviors
- Endpoint defense software, such as anti-malware and endpoint detection and response (EDR)
- Keyloggers and other infostealer malware, to intercept keyboard events in order to capture keystrokes and steal sensitive data
- Evasive malware, to prevent its code from being detected by endpoint defenses

In this section, first we'll discuss how hooking works in Windows before looking at some common hooking techniques. To wrap up the section and the chapter, we'll then look at how malware can implement and inject hooks into victim processes.

SetWindowsHookEx Hooking and Injection

One of the simplest ways to implement a hook is to use a Windows function designed specifically for that purpose: `SetWindowsHookEx`. This function allows the caller to specify which system events to hook, such as mouse or keyboard events, and which code to execute when an event occurs. However, `SetWindowsHookEx` can also be abused for DLL injection, as it accepts a DLL file as a parameter. The following simplified pseudocode illustrates how `SetWindowsHookEx` can be used to hook and stealthily run malicious code within the context of a victim process:

```
hmod = LoadLibrary("evil.dll");  
lpfn = GetProcAddress(hmod, "Function_1");  
idHook = "WH_MOUSE_LL";  
dwThreadId = 0;  
HHOOK hook = SetWindowsHookEx(idHook, lpfn, hmod, dwThreadId);
```

In this example, several variables are passed to `SetWindowsHookEx` as parameters. The `idHook` parameter can have a number of different values, each representing a system event to be monitored and hooked. For example, the values `WH_MOUSE` and `WH_MOUSE_LL` (the latter is shown here) will intercept mouse-related events such as clicks. Two other hooks commonly used by malware are `WH_KEYBOARD` and `WH_KEYBOARD_LL`, which intercept keyboard-related events.

The `hmod` parameter represents the name of the loaded module containing the malicious code that will be executed once a mouse event occurs. In the current example, this malware is calling `LoadLibrary`, which will load the malicious DLL (*evil.dll*) into the victim process. The `lpfn` parameter represents a pointer to code that will execute when a mouse event occurs. In this case, the malicious function to be executed is `Function_1`, which resides in *evil.dll*.

Finally, the `dwThreadId` parameter represents a specific thread ID to be monitored. If the program calling `SetWindowsHookEx` wishes to monitor only a single thread in a single application, it can set the thread ID here. If the calling program wishes to monitor all threads, it can set this parameter to 0.

In summary, the malicious *evil.dll* file will load and execute inside the victim process whenever a mouse event is detected. Since the malware could potentially hide and wait for a very specific event to occur, such as a certain number of mouse clicks or a particular key being pressed, this attack can be very stealthy. Note, however, that this technique is rather old, and modern versions of Windows can prevent such an attack, not to mention that this behavior is very suspicious and will likely trigger endpoint defenses. Furthermore, this specific example will almost certainly result in a very unstable system. (Loading a DLL every time there's a mouse event isn't a wise decision!) However, variations of this technique are still used in modern malware, so it's important to be aware of it.

Inline Hooking

One of the most common forms of user-space hooking is *inline hooking*, which relies on injecting code into a target process and simultaneously modifying the legitimate function being hooked to force a jump to the injected code. First, the process that wishes to hook another process (the target process) must inject code into it by using one of the previously described methods; DLL injection is one of the most common. Next, the first process modifies the function it wishes to hook within the target process's address space to point and jump to the injected hooking code. When the target process executes that function, control flow is transferred from the original code to the hooked code, which can then be used for monitoring, intercepting, or modifying calls to the original function. **Figure 12-22** illustrates the inline hooking technique.

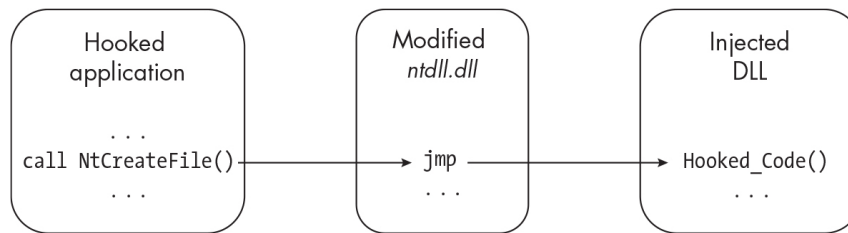


Figure 12-22: The inline API hooking technique

This application calls the WinAPI function `NtCreateFile`, which is located inside `ntdll.dll`. Normally, the `NtCreateFile` code inside `ntdll.dll` would then execute. However, `ntdll.dll` has been tampered with and its legitimate code has been overwritten with a jump instruction, so now the program will jump to and execute the hooking code instead. Once the malicious code is run, control flow is transferred back to `ntdll.dll` and finally returns to the original application.

Let's say our system has been infected with a malware sample that wishes to intercept and monitor all calls to `NtCreateFile` in a victim process (*firefox.exe*, for example). After injecting the malicious hooking DLL into *firefox.exe*, the malware must modify the `ntdll.dll` module in the victim process's address space. The malware locates the `NtCreateFile` function within `ntdll.dll` and overwrites the first 5 bytes to jump to the malicious injected DLL instead of executing the actual `NtCreateFile` code. (Overwriting 5 bytes is common because that's a typical size for jump instructions.) The first byte, `E9` (the jump instruction), is followed by 4 bytes representing the memory location to jump to. To modify the function code in the target process, the first process can call `WriteProcessMemory` OR `memcpy`. This code block shows the first bytes of the legitimate `NtCreateFile` function (in x64 assembly):

```
--snip--
mov r10, rcx
mov eax, 55
test byte ptr ds:[7FFE0308], 1
jne ntdll.7FFC61D3CB65
--snip--
```

After the malware modifies the `NtCreateFile` function, it might look like this; notice the jump to the malicious code:

```
--snip--
jmp hooked_code ; Jump to hooking code.
mov eax, 55
test byte ptr ds:[7FFE0308], 1
jne ntdll.7FFC61D3CB65
--snip--
```

Instead of a `jmp` statement, the jump to the hooked code could be accomplished with a `push` and `ret` instruction like so:

```
--snip--
push address_of_hooked_code ; Push the address of the hooked code to the stack.
ret                          ; Return (jump) to the hooked code.
--snip--
```

When installing an inline hook, the code author often will want the original function code to execute after the hooked code runs in order to avoid scrutiny from system users. Let's say that in our `NtCreateFile` example, the author wants to intercept calls to `NtCreateFile`, execute the hooked code, and then execute the original `NtCreateFile` code. This can be accomplished in the form of a *trampoline*, which is simply a jump back to the original function, as shown in [Figure 12-23](#).

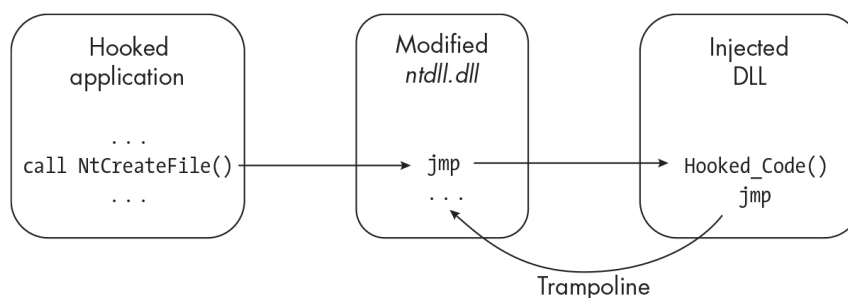


Figure 12-23: An inline hook trampoline

Trampolines are often implemented with an instruction such as `call NtCreateFile+5`. After the hooking code executes (inside the injected DLL, in this case), this instruction will transfer control flow back to the real `NtCreateFile`, skipping over the first 5 bytes where the inserted `jmp` statement resides in the modified `ntdll.dll` file.

When you're analyzing malware, there are three function calls that can hint at inline hooking. First, the malware calls `ReadProcessMemory` to read the first bytes of code in the function it wishes to hook inside the target process. Then, the malware calls `VirtualProtect` to modify the permissions of the target memory region and prepare for writing the jump instruction that will jump to the malicious code injected into the target process. Finally, the malware calls `WriteProcessMemory` to write the jump statement into the target function's code.

NOTE

Another user-space hooking technique is IAT hooking, which involves altering the import address table of the target process to point to the hooking code rather than to the original function code. However, IAT hooking isn't

often used by modern malware since it's easily detected by host-based defenses, so I won't go into more detail on this method.

Mitigations for Process and Hook Injection

As new injection and hooking techniques are used in malware or discovered by researchers and published, Microsoft counters them with built-in protections as much as possible. For example, *data execution prevention*, introduced in Windows XP, seeks to prevent malware from executing injected malicious code by marking memory regions as nonexecutable. *AppLocker*, introduced in Windows 7, prevents unauthorized executables (which can include injected DLLs) from executing. *Control Flow Guard*, released in Windows 8.1, is designed to detect whether malware modifies the code control flow of another process, which often takes place during process injection. And *Arbitrary Code Guard (ACG)*, added in Windows 10, seeks to prevent malware from modifying the code of legitimate processes, thus preventing certain types of process and hook injection.

In addition, complementary defenses such as EDR and anti-malware software can monitor suspicious processes and detect or prevent some of these injections. Keep in mind, however, that not all injection is malicious: plenty of benign, legitimate applications use injection for various reasons, so it can be difficult for endpoint defense products to determine the difference between good and evil. [Chapter 13](#) will explore some of these endpoint defenses and how malware attempts to bypass them.

Summary

In this chapter, we discussed some of the common techniques malware uses to inject and run code within the context of other processes in order to evade endpoint defenses and blend in with the environment in which it's running. We also covered some of the common hooks malware uses to hide its malicious behaviors or implement rootkit functionality. There are many types of injection and hooking techniques, and it's impossible to cover them all in this book. I've tried to focus on those that you're most likely to encounter in the wild, as well as some of the newer techniques that malware authors might add to their evasion toolboxes.

In the next chapter, we'll start looking at ways in which malware can circumvent endpoint and network defenses to execute and hide its malicious code in a well-protected environment.